

UNIVERSIDADE LICUNGO

**Faculdade de Ciências e Tecnologia
Curso de Licenciatura em Informática**

Cadeira: Laboratório I

Trabalho Prático Integrado

**Tema: Desenvolvimento de um Pequeno Projeto em Haskell
com Controlo de Versões e Documentação em LaTeX**

Discentes:

Latifo José Manuel Campos (Líder)
Adelcio Abudo Amudane
Domingos Raul João
Samuel Antonio Madonda
Sidney Nordino Damodar

Docente:

Mestre Geraldo Cardoso Sotaria

**Dondo
2026**

Conteúdo

1	Introdução	2
2	Objetivos	2
2.1	Objetivo Geral	2
2.2	Objetivos Específicos	2
3	Metodologia	3
3.1	Metodologia de Desenvolvimento	3
4	Desenvolvimento	3
5	Explicação do Código	4
5.1	Função Principal da Aplicação	5
5.2	Funcionamento da Função loop	6
5.3	Listagem dos Estudantes	7
5.4	Pesquisa de Estudantes	9
5.5	Síntese da Implementação	9
6	Resultados de Execução e Capturas de Ecrã	10
7	Conclusão	11

1 Introdução

A programação funcional constitui um dos principais paradigmas da programação de computadores e distingue-se pela construção de programas baseados na avaliação de funções matemáticas, evitando alterações de estado e dados mutáveis. Esta abordagem promove a criação de código mais previsível, modular e de fácil manutenção, sendo amplamente utilizada no ensino da Ciência da Computação e em aplicações que exigem elevado nível de fiabilidade (Bird, 2015).

Entre as linguagens que implementam este paradigma, o Haskell destaca-se por ser uma linguagem funcional pura, fortemente tipada e de avaliação preguiçosa (*lazy evaluation*). Estas características permitem desenvolver programas mais seguros, reduzindo a ocorrência de erros durante a compilação e incentivando uma organização lógica do código. Segundo Hudak (2000), o Haskell foi concebido para servir simultaneamente como linguagem de investigação e de ensino, tornando-se uma referência mundial na aprendizagem da programação funcional.

A realização deste trabalho permitiu integrar conhecimentos teóricos e práticos adquiridos ao longo da disciplina, promovendo não apenas o domínio da programação funcional, mas também o desenvolvimento de competências relacionadas com a utilização de ferramentas de apoio ao desenvolvimento de software. A divisão das tarefas entre os membros do grupo favoreceu a aprendizagem colaborativa, possibilitando que cada estudante contribuisse para diferentes fases do projeto, desde a implementação da aplicação até à elaboração da documentação técnica e validação dos resultados obtidos.

2 Objetivos

2.1 Objetivo Geral

Desenvolver uma aplicação interativa para a gestão de uma lista de estudantes utilizando a linguagem de programação funcional Haskell, aplicando os conceitos fundamentais da programação funcional e utilizando ferramentas de desenvolvimento como Linux, Git, GitHub e LaTeX.

2.2 Objetivos Específicos

- Implementar uma aplicação que permita inserir estudantes.
- Listar os estudantes registados.
- Contar o número de estudantes existentes.
- Pesquisar estudantes pelo nome.
- Aplicar listas e recursividade na implementação da aplicação.
- Utilizar o Git e o GitHub para o controlo de versões.
- Utilizar o Linux como ambiente de desenvolvimento.

- Elaborar o relatório técnico em LaTeX.
- Testar e validar o funcionamento da aplicação.

3 Metodologia

3.1 Metodologia de Desenvolvimento

Para o desenvolvimento deste projeto foi adotada uma metodologia prática e incremental. Inicialmente, foi realizada a análise dos requisitos do sistema, seguida do planeamento das atividades e da implementação da aplicação em Haskell.

O desenvolvimento decorreu em ambiente Linux, utilizando o compilador GHC para compilar e executar o programa. O controlo de versões foi efetuado com o Git, enquanto o GitHub foi utilizado para o alojamento e sincronização do repositório.

Após a implementação, foram realizados testes funcionais para validar as operações de inserção, listagem, contagem e pesquisa de estudantes. Por fim, a documentação do projeto foi elaborada em LaTeX, garantindo uma apresentação organizada e de acordo com as normas académicas.

4 Desenvolvimento

O desenvolvimento da aplicação foi realizado na linguagem de programação Haskell, tendo como objetivo implementar um sistema simples para a gestão de uma lista de estudantes. Durante a implementação, foram aplicados os principais conceitos da programação funcional estudados na disciplina, como a utilização de listas, recursividade e funções pre-definidas.

A aplicação foi estruturada para funcionar através de um menu interativo apresentado na consola, permitindo ao utilizador selecionar as diferentes operações disponíveis. Inicialmente, o sistema cria uma lista vazia de estudantes e mantém o programa em execução até que seja selecionada a opção de saída.

As funcionalidades implementadas incluem a inserção de novos estudantes, a listagem dos estudantes registados, a contagem do número total de estudantes e a pesquisa de estudantes pelo nome. Cada operação foi desenvolvida de forma independente e integrada posteriormente na aplicação principal, garantindo uma melhor organização do código.

Durante o desenvolvimento, foram utilizados o sistema operativo Linux para compilação e execução da aplicação, o Git para o controlo de versões, o GitHub para o alojamento do repositório e o LaTeX para a elaboração da documentação técnica. Estas ferramentas permitiram organizar o trabalho, acompanhar a evolução do projeto e facilitar a colaboração entre os membros do grupo.

Após a implementação de todas as funcionalidades, foram realizados testes funcionais para verificar o correto comportamento da aplicação. Os resultados demonstraram que o sistema executa corretamente todas as operações previstas, respondendo de forma adequada às ações do utilizador e cumprindo os requisitos definidos no enunciado do projeto.

```
hostydomain@DESKTOP-SEGTPPR: ~/ProjetoHaskell
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ mkdir -p ~/ProjetoHaskell/Relatorio && cd ~/ProjetoHaskell
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ pwd
~/ProjetoHaskell
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ echo "Ficheiro temporario de teste" > teste.txt
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ cp teste.txt copia_teste.txt
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ ls -la
total 20
drwxr-xr-x 3 hostydomain hostydomain 4096 Jul 10 08:02 .
drwxr-xr-x 5 hostydomain hostydomain 4096 Jul 10 07:37 ..
drwxr-xr-x 2 hostydomain hostydomain 4096 Jul 10 07:37 Relatorio
-rw-r--r-- 1 hostydomain hostydomain 29 Jul 10 08:02 copia_teste.txt
-rw-r--r-- 1 hostydomain hostydomain 29 Jul 10 08:02 teste.txt
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ mv copia_teste.txt backup.txt
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ rm teste.txt backup.txt
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ ls -la
total 12
drwxr-xr-x 3 hostydomain hostydomain 4096 Jul 10 08:02 .
drwxr-xr-x 5 hostydomain hostydomain 4096 Jul 10 07:37 ..
drwxr-xr-x 2 hostydomain hostydomain 4096 Jul 10 07:37 Relatorio
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$
```

Figura 1: Execução prática de comandos Linux necessários para a administração do projeto no terminal.

```
hostydomain@DESKTOP-SEGTPPR: ~/ProjetoHaskell
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ # 3. Criar o Commit 8 final contendo o relatório técnico e os nomes do grupo.
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ git commit -m "Commit 8: Relatório tecnico final e identificacao dos membros"
[master f018607] Commit 8: Relatório tecnico final e identificacao dos membros
3 files changed, 221 insertions(+)
create mode 100644 Relatorio/Relatorio.pdf
create mode 100644 Relatorio/Relatorio.tex
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ # 4. Atualizar o GitHub substituindo a versão anterior e forçando a contagem exata de 8 commits
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ git push origin master --force
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 4 threads
Compressing objects: 100% (26/26), done.
Writing objects: 100% (27/27), 108.56 KiB | 13.57 MiB/s, done.
Total 27 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/Claudio8472/ProjetoHaskell.git
+ 01c8db3...f018607 master -> master (forced update)
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$ git log --oneline
f018607 (HEAD -> master, origin/master) Commit 8: Relatório tecnico final e identificacao dos membros
3f4819d (feature-pesquisa) Commit 7: Implementada a pesquisa de estudantes no ramo feature-pesquisa
40a862f Commit 6: Adicionadas funcionalidades de listagem e contagem
18a5deb Commit 5: Implementada a funcionalidade de Insercao de estudantes
d0672ab Commit 4: Implementado o esqueleto do menu recursivo
469a4e4 Commit 3: Criacao do ficheiro Main.hs inicial
93a561c Commit 2: Adicionado README.md inicial
9a0a62f Commit 1: Inicializacao do repositorio e criacao do .gitignore
hostydomain@DESKTOP-SEGTPPR:~/ProjetoHaskell$
```

Figura 2: Visualização da árvore de 8 commits e ramos controlados através do Git.

5 Explicação do Código

A aplicação desenvolvida foi implementada utilizando a linguagem de programação funcional Haskell, uma linguagem reconhecida pela sua forte tipagem, imutabilidade dos dados e utilização de funções puras. Diferentemente das linguagens imperativas, em que o estado do programa é alterado continuamente através da modificação de variáveis, o Haskell privilegia a utilização de funções que recebem dados como entrada e produzem novos resultados sem alterar os valores existentes. Esta abordagem torna o código mais

seguro, previsível e fácil de manter (Bird, 2015).

O programa desenvolvido tem como finalidade gerir uma lista de estudantes através de uma interface interativa executada na consola. Para isso, foram implementadas funcionalidades que permitem inserir novos estudantes, listar os estudantes registados, contar o número total de elementos existentes e pesquisar estudantes pelo nome.

A estrutura da aplicação foi organizada de forma simples e modular, dividindo as responsabilidades por diferentes funções. Esta organização facilita a leitura do código, reduz a duplicação de instruções e permite futuras melhorias ou a inclusão de novas funcionalidades sem comprometer a estrutura principal do programa.

Durante a implementação foram utilizados diversos conceitos fundamentais da programação funcional, entre os quais se destacam:

- utilização de listas como principal estrutura de armazenamento de dados;
- recursividade para controlar o fluxo da aplicação;
- funções predefinidas da biblioteca padrão do Haskell;
- operações de entrada e saída através do tipo IO;
- estruturas condicionais para o controlo das opções do menu.

Each of these components plays an important role in the operation of the application and is described in the following sections.

5.1 Função Principal da Aplicação

A execução da aplicação inicia-se através da função `main`, que constitui o ponto de entrada de qualquer programa desenvolvido em Haskell. É nesta função que o sistema realiza a inicialização da aplicação, apresenta uma mensagem de boas-vindas ao utilizador e inicia o ciclo principal responsável pela interação com o sistema.

O código correspondente encontra-se apresentado a seguir:

```
1 main :: IO ()
2 main = do
3     putStrLn "Bem-vindo ao Sistema de Gestao de Estudantes!"
4     loop []
```

A declaração `main :: IO ()` indica que a função `main` pertence ao tipo IO, o que significa que executa operações de entrada e saída de dados. Em Haskell, qualquer interação com o utilizador, como escrever mensagens no ecrã ou ler informação introduzida pelo teclado, deve ser realizada dentro do tipo IO. Esta separação entre funções puras e operações de entrada e saída constitui uma das principais características da linguagem, permitindo preservar a previsibilidade das restantes funções do programa.

Em seguida, é utilizada a instrução `putStrLn` que é responsável por apresentar uma mensagem no terminal, seguida de uma mudança de linha (*newline*). Neste caso, a mensagem tem como objetivo informar o utilizador de que a aplicação foi iniciada com sucesso.

Após a apresentação da mensagem inicial, a função `main` executa a instrução `loop []`. Esta chamada inicia o funcionamento da aplicação propriamente dita. O parâmetro

[] representa uma lista vazia, indicando que, no momento em que o programa é iniciado, ainda não existe qualquer estudante registrado.

À medida que o utilizador adiciona novos estudantes, esta lista deixa de estar vazia e passa a armazenar os nomes introduzidos. Como o Haskell trabalha com estruturas imutáveis, cada alteração resulta na criação de uma nova lista, que é transmitida novamente para a função `loop`. Desta forma, o estado da aplicação é preservado sem necessidade de modificar diretamente os dados existentes.

5.2 Funcionamento da Função `loop`

A função `loop` constitui o núcleo da aplicação, sendo responsável por manter o programa em execução e permitir a interação contínua entre o utilizador e o sistema. É através desta função que o menu principal é apresentado, as opções introduzidas pelo utilizador são processadas e as diferentes funcionalidades da aplicação são executadas.

Ao contrário de muitas linguagens de programação imperativas, onde a repetição de instruções é normalmente realizada através de ciclos (*loops*) como `for` e `while`, em Haskell este comportamento é obtido recorrendo à recursividade. Assim, após concluir cada operação, a função `loop` chama novamente a si própria, garantindo que o menu seja apresentado repetidamente até que o utilizador escolha terminar a aplicação.

A estrutura simplificada da função é apresentada a seguir:

```
1 loop :: [String] -> IO ()
2 loop estudantes = do
```

A declaração acima indica que a função `loop` recebe como argumento uma lista de cadeias de caracteres (`[String]`), correspondente aos nomes dos estudantes registados. Como resultado, devolve uma operação do tipo `IO`, uma vez que envolve interação com o utilizador.

O parâmetro `estudantes` representa o estado atual da aplicação. Sempre que um novo estudante é inserido, uma nova lista é criada e passada novamente para a função `loop`.

Apresentação do Menu

Sempre que a função é executada, o primeiro passo consiste em apresentar o menu principal da aplicação:

```
1 putStrLn "\n==== MENU ===="
2 putStrLn "1- Inserir estudante"
3 putStrLn "2- Listar estudantes"
4 putStrLn "3- Contar estudantes"
5 putStrLn "4- Procurar estudante"
6 putStrLn "5- Sair"
```

Leitura da Opção Escolhida

Depois de apresentar o menu, o sistema solicita ao utilizador a introdução de uma opção:

```

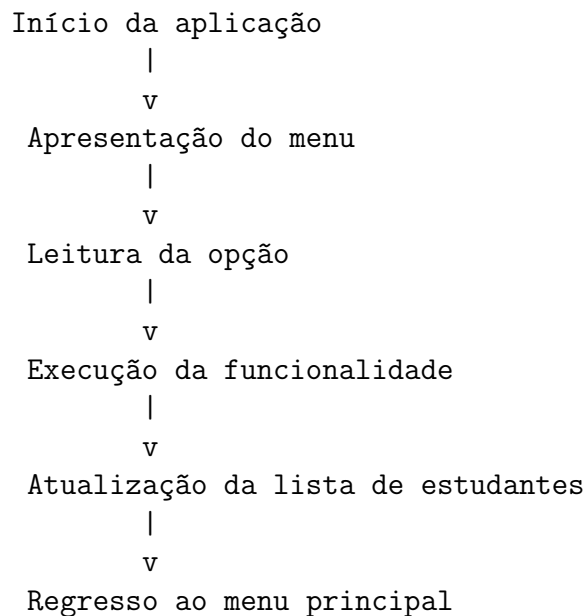
1 putStr "Escolha uma opcao: "
2 hFlush stdout
3 opcao <- getLine

```

A função `putStr` apresenta a mensagem sem mudar de linha, permitindo que o utilizador escreva imediatamente a sua resposta. Em seguida, é utilizada a função `hFlush stdout`, cuja finalidade é garantir que a mensagem seja apresentada imediatamente no ecrã. Por fim, a função `getLine` lê a informação introduzida pelo utilizador através do teclado.

Fluxo de Execução da Aplicação

Após a leitura da opção escolhida, o programa verifica qual a operação correspondente e executa a funcionalidade adequada. O fluxo de execução pode ser representado pelo seguinte esquema:



5.3 Listagem dos Estudantes

A funcionalidade de listagem permite apresentar ao utilizador todos os estudantes previamente registados no sistema. Esta operação demonstra a utilização de funções recursivas em Haskell, uma vez que a linguagem não utiliza ciclos tradicionais como `for` ou `while`, comuns em linguagens imperativas. Em programação funcional, a repetição de operações sobre estruturas de dados é normalmente realizada através de recursividade ou funções de ordem superior (Hutton, 2016).

A implementação associada à opção "2" do menu segue a seguinte lógica:

```

1 "2" -> do
2     putStrLn "\n--- Lista de Estudantes ---"
3     listarEstudantes estudantes
4     loop estudantes

```


A função auxiliar responsável pela apresentação dos dados pode ser definida da seguinte forma:

```
1 listarEstudantes :: [String] -> IO ()
2 listarEstudantes [] = putStrLn "Nenhum estudante registrado."
3 listarEstudantes (x:xs) = do
4     putStrLn x
5     listarEstudantes xs
```

Recursividade e Correspondência de Padrões

A função `listarEstudantes` utiliza correspondência de padrões (*pattern matching*) para analisar o conteúdo da lista recebida como argumento. O primeiro padrão `listarEstudantes []` representa o caso base da recursividade, ou seja, quando a lista está vazia. O segundo padrão `listarEstudantes (x:xs)` representa uma lista não vazia, dividida em duas partes:

- **x**: representa o primeiro elemento da lista (cabeça);
- **xs**: representa o restante dos elementos (cauda).

Ausência de Estruturas de Repetição Imperativas

O processo ocorre da seguinte forma:

1. A função recebe a lista completa de estudantes;
2. O primeiro nome é apresentado através de `putStrLn`;
3. A função chama-se novamente passando apenas o restante da lista;
4. O processo continua até atingir uma lista vazia.

Este comportamento pode ser representado de forma abstrata como:

```
listarEstudantes [João, Ana, Pedro]
-> imprime João -> chama listarEstudantes [Ana, Pedro]
-> imprime Ana -> chama listarEstudantes [Pedro]
-> imprime Pedro -> chama listarEstudantes []
-> termina
```

Eficiência da Operação

A complexidade desta operação é $O(N)$, onde N representa o número total de estudantes existentes na lista. Isto acontece porque todos os elementos precisam obrigatoriamente de ser percorridos para serem apresentados no terminal.

5.4 Pesquisa de Estudantes

A funcionalidade de pesquisa permite verificar se um determinado estudante se encontra registrado na lista de dados da aplicação. Esta operação foi associada à opção "4" do menu principal, permitindo ao utilizador introduzir um nome e obter uma resposta imediata sobre a existência desse estudante no sistema.

O bloco de código responsável por esta funcionalidade é apresentado abaixo:

```
1 "4" -> do
2   putStr "Introduza o nome a procurar: "
3   hFlush stdout
4   nomePesquisa <- getLine
5
6   if nomePesquisa `elem` estudantes
7     then putStrLn "Estudante encontrado no sistema."
8     else putStrLn "Estudante nao encontrado."
9
10  loop estudantes
```

Utilização do Operador elem

A verificação da existência de um estudante é realizada através da função predefinida elem. Esta função recebe um elemento e uma lista, retornando um valor lógico do tipo Bool, podendo assumir os valores True ou False. No contexto da aplicação, o operador infixo nomePesquisa `elem` estudantes permite uma leitura mais natural da condição.

Pesquisa Sequencial em Listas

Como a estrutura utilizada para armazenar os estudantes é uma lista simplesmente ligada, a operação de pesquisa é realizada de forma sequencial. No pior caso, a complexidade temporal desta operação é $O(N)$, onde N representa o número total de estudantes armazenados (Hutton, 2016).

5.5 Síntese da Implementação

A Tabela 1 apresenta um resumo dos principais conceitos de programação funcional aplicados no desenvolvimento da aplicação de gestão de estudantes:

A implementação desenvolvida demonstra a aplicação prática dos principais conceitos estudados na disciplina de Programação Funcional. Através da utilização de listas imutáveis, funções recursivas, avaliação preguiçosa e operações monádicas, foi possível construir uma aplicação simples, modular e de fácil manutenção.

Conceito Aplicado	Função / Operador	Objetivo no Projeto
Imutabilidade	++ (Concatenação)	Criar uma nova lista durante a inserção de estudantes sem modificar diretamente os dados existentes (Bird, 2015).
Recursividade	loop estudantes	Manter o programa em execução contínua sem recorrer a estruturas imperativas como while ou for.
Efeitos Colaterais	Mónada IO / mapM_	Controlar operações de entrada e saída, separando a lógica funcional das interações com o utilizador (Thompson, 2011).
Lazy Evaluation	zip [1..]	Gerar índices automaticamente apenas quando necessários, evitando desperdício de memória.
Pattern Matching	case ... of	Controlar diferentes opções do menu através da correspondência de padrões, tornando o código mais organizado e legível.

Tabela 1: Síntese dos conceitos de programação funcional aplicados.

6 Resultados de Execução e Capturas de Ecrã

As validações e testes foram executados na plataforma Linux utilizando o compilador GHC [1].

```

hugobonin@DESKTOP-SEGTB9R: ~/ProjetoHaskell
1 - Listar estudantes
2 - Contar estudantes
3 - Procurar estudante
4 - Sair
Escolha uma opcao: 1
Introduza o nome do estudante: Maria
Estudante adicionado com sucesso!

===== MENU =====
1 - Inserir estudante
2 - Listar estudantes
3 - Contar estudantes
4 - Procurar estudante
5 - Sair
Escolha uma opcao: 1
Introduza o nome do estudante: Joao
Estudante adicionado com sucesso!

===== MENU =====
1 - Inserir estudante
2 - Listar estudantes
3 - Contar estudantes
4 - Procurar estudante
5 - Sair
Escolha uma opcao: 2

--- Lista de Estudantes ---
1. Maria
2. Joao

===== MENU =====
1 - Inserir estudante
2 - Listar estudantes
3 - Contar estudantes
4 - Procurar estudante
5 - Sair
Escolha uma opcao:

```

Figura 3: Interface de execução interativa da aplicação de Gestão de Estudantes no Linux.

Como demonstrado na Figura 3, o comportamento recursivo assegura a correta manutenção desde dados do estado da lista em memória até ao encerramento da sessão de teste.

7 Conclusão

O desenvolvimento deste projeto integrado proporcionou ao Grupo 2 da Universidade Licungo uma compreensão sólida e prática sobre os pilares da programação funcional e as metodologias de engenharia de software modernas. A imutabilidade e a pureza de funções em Haskell demonstraram ser conceitos de grande valia para o desenvolvimento de código fiável e livre de efeitos colaterais.

Adicionalmente, a utilização conjunta de ferramentas como Git, GitHub e o compilador LaTeX [2] garantiu um fluxo de trabalho estruturado e uniforme aos discentes da Universidade Licungo, assegurando a entrega de um produto final que cumpre integralmente os requisitos curriculares estipulados.

Referências

- [1] GHC Team, *The Glasgow Haskell Compiler Users Guide*, disponível em <https://www.haskell.org/ghc/>, acesso em 2026.
- [2] Lamport, L., *LaTeX: A Document Preparation System*, Addison-Wesley, 2.^a Edição, 1994.
- [3] Hutton, G., *Programming in Haskell*, Cambridge University Press, 2.^a Edição, 2016.
- [4] Bird, R., *Thinking Functionally with Haskell*, Cambridge University Press, 2015.
- [5] Thompson, S., *Haskell: The Craft of Functional Programming*, Addison-Wesley, 3.^a Edição, 2011.